

Te dejo el documento técnico v1.0 con el alcance que definimos. La idea es que este sea nuestro punto de referencia y que no agreguemos funciones porque sí mientras programamos.

Documento técnico — Gestor de Territorios Telefónicos

MVP v1.0

1. Objetivo

Crear una webapp sencilla para reemplazar y mejorar el manejo actual de territorios telefónicos que hoy se realiza mediante varios archivos Excel.

La aplicación permitirá:

Importar las bases existentes.

Organizar los datos por territorio.

Visualizar cada territorio de forma independiente.

Consultar y modificar sus registros telefónicos.

Asignar territorios a responsables.

Registrar cuándo se asignó y cuándo finalizó un trabajo.

Mantener un historial.

Generar una lista de un territorio en PNG y PDF.

Exportar información cuando sea necesario.

La primera versión funcionará localmente en la computadora (localhost).

El objetivo inicial no es crear un sistema empresarial robusto, sino una aplicación funcional que además permita aprender progresivamente desarrollo web.

2. Alcance del MVP

La aplicación deberá permitir:

Importar una base telefónica desde Excel.

Crear/actualizar los territorios a partir de esa información.

Ver el listado de territorios.

Buscar un territorio.

Abrir un territorio.

Ver todos sus registros telefónicos.

Modificar: Observaciones

No llamar

Funcionan

Notas internas

Administrar responsables.

Asignar un territorio a un responsable.

Impedir que un territorio ya asignado sea asignado simultáneamente a otra persona.

Finalizar un trabajo.

Registrar el historial de asignaciones.

Registrar cambios importantes realizados sobre los datos.

Generar una lista del territorio en PNG.

Generar una versión PDF para imprimir.

Exportar datos a Excel.

3. Fuera del MVP

Por ahora NO se implementará:

Login.

Contraseñas.

Usuarios con cuentas individuales.

Roles/permisos avanzados.
Aplicación móvil.
API pública.
Integraciones externas.
Notificaciones.
Hosting.
Dominio.
PostgreSQL.
Docker.
Sistema de despliegue.
Estadísticas avanzadas.
Dashboard complejo.
Todo esto podrá evaluarse posteriormente.

4. Stack tecnológico

Backend

Python + Flask

Responsabilidades:

Recibir solicitudes del frontend.

Consultar y modificar SQLite.

Procesar archivos Excel.

Generar exportaciones.

Generar PNG/PDF.

Aplicar las reglas de negocio.

Frontend

HTML + CSS + JavaScript

Responsabilidades:

Mostrar la información.

Formularios.

Tablas.

Búsquedas.

Botones y acciones.

Interacciones dinámicas.

Comunicación con Flask cuando sea necesario.

No se utilizará React inicialmente.

Base de datos

SQLite

Adecuada para la primera versión porque:

Es sencilla.

No necesita servidor.

Está integrada fácilmente con Python.

Permite migrar posteriormente a una base más robusta.

Excel

Python utilizando principalmente:

pandas

openpyxl

5. Modelo conceptual

La unidad principal del sistema es el:

Territorio

Ejemplo:

Territorio 404

Un territorio contiene muchos registros telefónicos.

Además, puede tener múltiples asignaciones a lo largo del tiempo.

Territorio 404



6. Entidades principales

6.1 Territorios

Representan las unidades de trabajo.

Campos iniciales:

id

numero

estado

Estados:

Disponible

En trabajo

Un territorio finalizado vuelve a estar disponible.

6.2 Registros telefónicos

Representan cada fila de la base telefónica.

Campos:

id

territorio_id

direccion

telefono

observaciones

no_llamar

funcionan

notas_internas

Identificación única

Un registro se considera único mediante:

Territorio + Dirección + Teléfono

Por ejemplo:

404 + Mitre 46 + 4292-9527

6.3 Responsables

Personas a las que se pueden asignar territorios.

Campos:

id

nombre

activo

No son usuarios del sistema.

No tienen contraseña.

Ejemplo:

Vicenzo

Jorge

Sulema

El administrador podrá agregar o desactivar responsables.

7. Asignaciones

Una asignación representa el período durante el cual una persona trabaja un territorio.

Campos:

id

territorio_id

responsable_id

fecha_asignado

fecha_finalizacion

detalles

Ejemplo:

Territorio: 404

Responsable: Vicenzo

Asignado: 13/08/2026

Finalizado: 15/08/2026

Cuando se asigna un territorio:

Disponible

↓

En trabajo

Cuando se finaliza:

En trabajo

↓

Disponible

La asignación anterior permanece guardada.

8. Historial / actividad

Se debe conservar un registro de las acciones relevantes.

Entidad:

actividad

Campos iniciales:

id

territorio_id

registro_id

responsable_id

tipo

descripcion

fecha

Ejemplo:

13/08/2026 10:32

Territorio: 404

Responsable: Vicenzo

Se modificó "Funcionan"

Teléfono: 4292-9527

Valor anterior: vacío

Nuevo valor: Sí

La implementación exacta de todos los tipos de actividad podrá evolucionar durante el desarrollo.

La prioridad es no perder información importante.

9. Datos provenientes de Excel

Base telefónica

La aplicación debe poder importar archivos con columnas equivalentes a:

Territorio

Dirección

Teléfono

Observaciones

No llamar

Funcionan

Notas internas

Valores de Funcionan

Puede ser:

Sí

No

Vacío

Internamente puede representarse como:

true

false

null

No llamar

Es un valor booleano:

Sí

No

10. Importación

La importación debe ser segura.

Si un registro no existe:

→ Crear

Si ya existe:

→ Actualizar sus datos

La aplicación NO debe crear duplicados cuando encuentre nuevamente:

Territorio + Dirección + Teléfono

Registros ausentes

Si un registro existente no aparece en un nuevo Excel:

NO se debe eliminar automáticamente.

Esto evita pérdida accidental de información.

Posteriormente podemos implementar un sistema de comparación/importación más avanzado.

11. Importación de asignaciones

También existe una estructura de Excel relacionada con:

Territorio

Responsable

Fecha asignado

Fecha finalización

Y una estructura histórica con:

Territorio

Responsable

Fecha asignado

Fecha completado

Detalles

La aplicación deberá poder utilizar estos datos para cargar el historial existente.

No necesariamente se deben conservar las hojas de Excel como estructura interna.

Los datos serán transformados a las entidades de SQLite.

12. Pantallas principales

12.1 Inicio

Objetivo: visualizar rápidamente los territorios.

Debe mostrar:

Territorio

Responsable

Estado

Ejemplo:

404 Vincenzo En trabajo

401 Vincenzo Disponible

315 Jorge Disponible

Debe incluir:

búsqueda;

acceso al territorio;

importación.

13. Vista de territorio

Al abrir:

Territorio 404

se mostrará:

Responsable: Vincenzo

Fecha asignado: 13/08/2026

Estado: En trabajo

Acciones:

Generar PNG

Generar PDF

Finalizar territorio

Ver historial

Y debajo los registros:

Dirección internas	Teléfono	No llamar	Funcionan	Observaciones	Notas
-----------------------	----------	-----------	-----------	---------------	-------

Los campos modificables son:

Observaciones

No llamar

Funcionan

Notas internas

14. Asignación

Desde el territorio:

Territorio 404

Responsable:

[Vincenzo ▼]

[Asignar]

Al confirmar:

territorio.estado = "En trabajo"

y se crea una nueva asignación.

Regla

Un territorio En trabajo no puede recibir otra asignación simultáneamente.

15. Finalización

Desde un territorio activo:

[Finalizar territorio]

Se registra:

fecha_finalizacion

La asignación queda cerrada.

El territorio pasa a:

Disponible

Y podrá volver a asignarse posteriormente.

16. Generación de lista

La aplicación debe poder generar una representación del territorio para entregar al responsable.

Contenido mínimo:

Territorio N° 404

Teléfono | No llamar | Observaciones

La información debe generarse desde los datos actuales del territorio.

Formatos

PNG

Para compartir digitalmente.

PDF

Para imprimir.

No se creará una segunda base de datos para las listas.

La lista es simplemente una representación/exportación del territorio actual.

17. Exportación

Se podrá exportar información nuevamente a Excel.

Inicialmente:

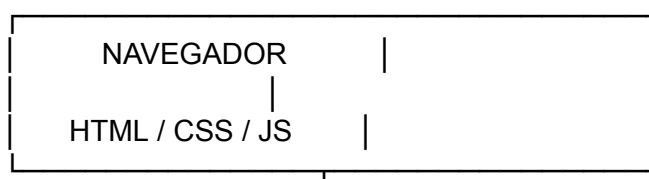
Base completa.

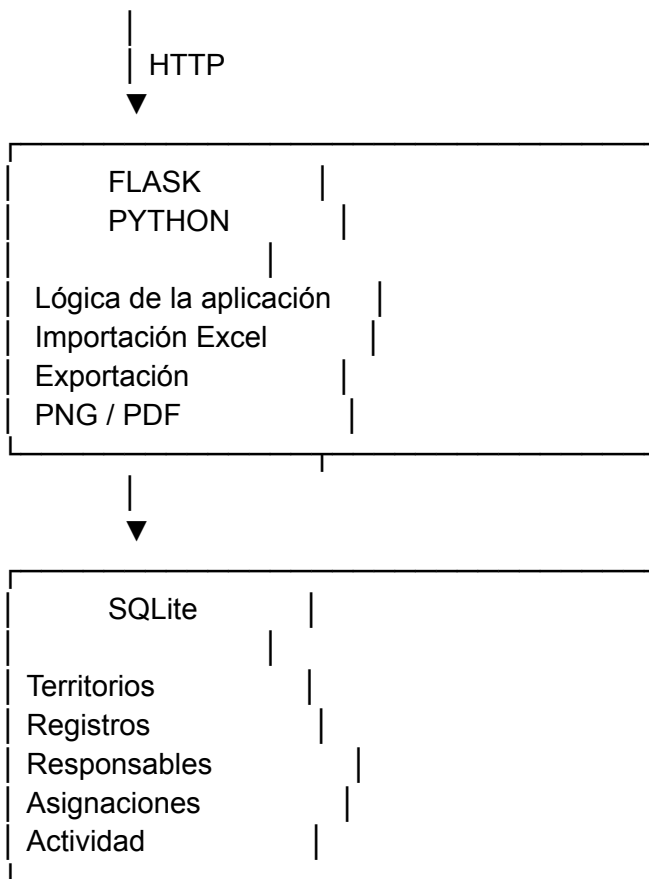
Territorio seleccionado.

Más adelante podrán agregarse filtros y exportaciones específicas.

18. Arquitectura

La arquitectura inicial será:

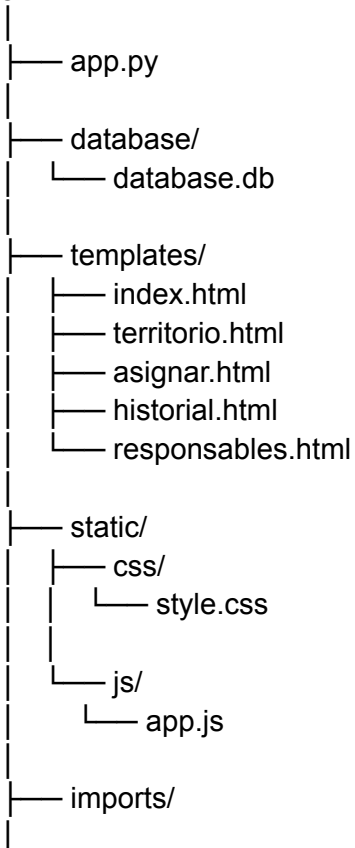




19. Estructura inicial del proyecto

Propuesta:

gestor_territorios/




```
|— exports/  
|  
|— requirements.txt
```

Esta estructura puede cambiar durante el desarrollo. No debe considerarse definitiva.

20. Orden de implementación

No vamos a construir todo de una vez.

Etapas 1 — Aplicación mínima

Crear:

Python

Flask

HTML

CSS

JavaScript

Objetivo:

Poder abrir localhost y ver nuestra primera página.

Etapas 2 — SQLite

Crear la base:

territorios

registros

responsables

asignaciones

actividad

Objetivo:

La aplicación ya tiene dónde guardar información.

Etapas 3 — Territorios

Implementar:

listado;

búsqueda;

apertura de territorio.

Objetivo:

Poder navegar por la estructura.

Etapas 4 — Registros

Implementar:

visualización;

edición;

guardado.

Objetivo:

Poder reemplazar la edición cotidiana que hoy hacen en Excel.

Etapas 5 — Importación

Implementar:

Excel

↓

Python

↓

Validación



SQLite

Objetivo:

Poder cargar la base existente.

Etapa 6 — Responsables y asignaciones

Implementar:

responsables;

asignación;

estado;

finalización.

Etapa 7 — Historial

Implementar:

historial de asignaciones;

actividad relevante.

Etapa 8 — PNG/PDF

Implementar la generación de:

lista PNG;

lista PDF.

Etapa 9 — Exportación

Implementar nuevamente:

SQLite → Excel

21. Criterio de éxito del MVP

Consideraremos que la primera versión funciona cuando podamos hacer esto:

1. Abrir la aplicación



2. Importar nuestro Excel



3. Ver los territorios



4. Buscar 404



5. Abrir 404



6. Ver todos sus teléfonos



7. Modificar un registro



8. Guardar



9. Asignar 404 a Vincenzo



10. Generar PNG/PDF



11. Finalizar 404



12. Volver a verlo como Disponible



13. Consultar su historial

Si todo eso funciona, tenemos una primera versión realmente útil.

22. Filosofía del proyecto

Hay una regla que propongo mantener durante todo el desarrollo:

Primero funcional, después bonito, después robusto.

No vamos a intentar construir la versión definitiva desde el principio.

Primero queremos demostrar que:

los datos entran → se organizan → se pueden trabajar → se registran → se pueden sacar.

Después mejoramos diseño, experiencia de usuario, rendimiento, seguridad y finalmente publicación web.

Primer objetivo concreto

Nuestro primer paso de programación va a ser extremadamente pequeño:

gestor_territorios/

└─ app.py

Levantar Flask y conseguir:

<http://localhost:5000>

mostrando una primera pantalla.

Después de eso recién creamos SQLite.

Así vas a poder entender qué está haciendo cada parte en lugar de recibir un proyecto gigante de golpe.